ALU Link

Connecting ALU Students with Startup Internship Opportunities

Technical Report

Prepared by: Lydivine

Course: Individual Assignment 2 - Final Flutter Project

Institution: African Leadership University (ALU)

Repository: (add your GitHub link here)

Abstract

ALU Link is a mobile application built with Flutter and Firebase that connects African Leadership University (ALU) students seeking internship experience with student-led startups and early-stage ventures inside the ALU ecosystem. The platform allows verified startups to post internship and volunteer opportunities across categories such as software development, design, marketing, and business analysis, while students can search, filter, bookmark, and apply to opportunities and track the status of every application in real time. This report documents the system architecture, Firebase backend structure, state management approach, key workflows, UI and UX reasoning, scalability considerations, testing strategy, challenges encountered, and planned future improvements for the platform.

1. Introduction and Problem Statement

Many ALU students struggle to secure internships at established organizations because of limited openings, strict experience requirements, and heavy competition. At the same time, several student-run startups on campus need help in areas such as software development, design, marketing, operations, research, business analysis, content creation, and community management, but have no simple way to reach students who are willing to help. ALU Link addresses this gap directly: it gives startups a place to post real opportunities, and gives students a trustworthy place to discover and apply for them, without needing an existing personal network on campus.

A key design decision was to not build a generic job board. Because anyone could claim to run a startup, the platform requires every startup profile to be reviewed and approved by an admin before its opportunities become visible to students. This keeps the platform specific to the ALU ecosystem and protects students from applying to accounts that are not real, recognized ventures.

2. System Architecture

The application follows a simple three-layer architecture: a UI layer made of Flutter screens grouped by user role (authentication, student, startup, admin), a state management layer built with the provider package, and a backend layer provided entirely by Firebase (Authentication and Cloud Firestore). Screens never talk to Firebase directly; instead they read and call methods on provider classes, which is what keeps the UI code simple and makes the data layer easy to change later without rewriting every screen.

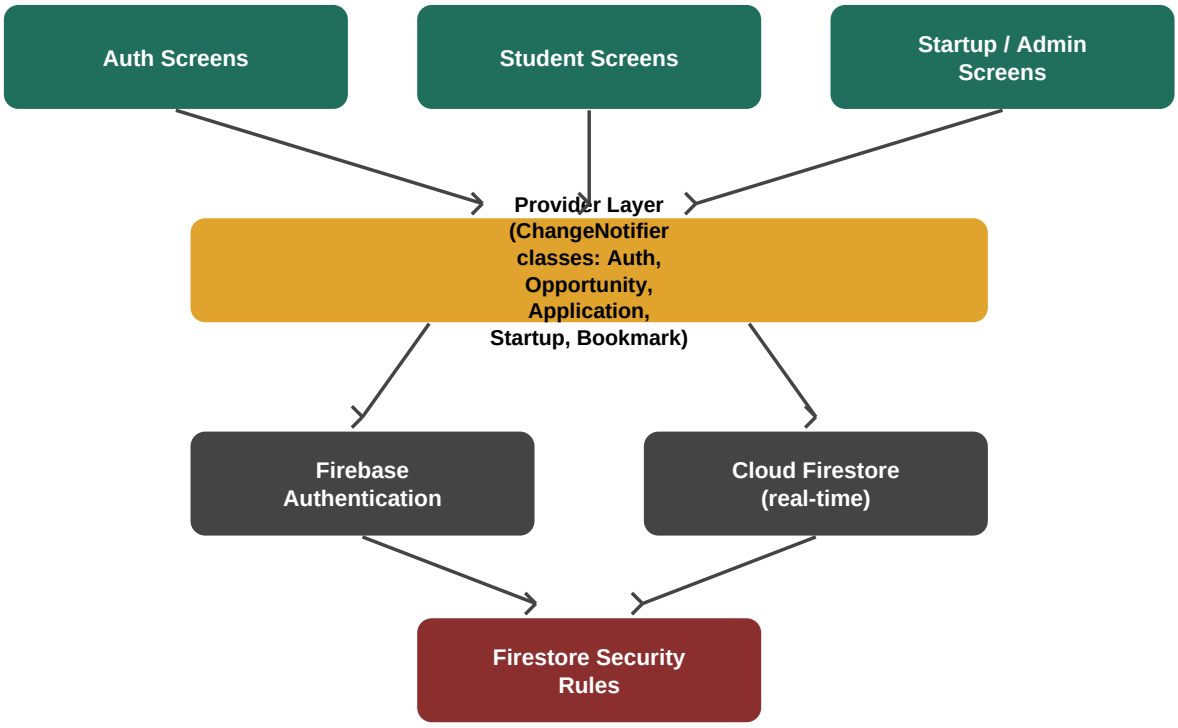


Figure 1. High level architecture of ALU Link.

Each provider is a ChangeNotifier: AuthProvider manages the signed-in user and their role, OpportunityProvider keeps a live list of opportunities and the current search/filter state, ApplicationProvider handles submitting and tracking applications, StartupProvider manages startup profile creation and the admin verification queue, and BookmarkProvider manages saved opportunities. Screens use context.watch to rebuild when data changes and context.read to call methods without rebuilding, which is the standard provider pattern.

2.1 Folder Structure

lib/models holds plain data classes with fromMap and toMap methods for converting to and from Firestore documents. lib/providers holds the five ChangeNotifier classes described above. lib/screens is split into auth, student, startup, and admin subfolders so that each role's screens are easy to find. lib/widgets holds small reusable pieces, such as the opportunity card shown in both the student browse list and the startup's own opportunity list.

3. Firebase Backend Structure

The backend uses two Firebase products: Firebase Authentication for email and password sign in, and Cloud Firestore as the main database. Firestore was chosen over a traditional REST backend because its snapshots() streams give the app real-time updates for free, which directly satisfies the requirement for dynamic, live data (for example, a student sees their application move from pending to accepted without refreshing the screen).

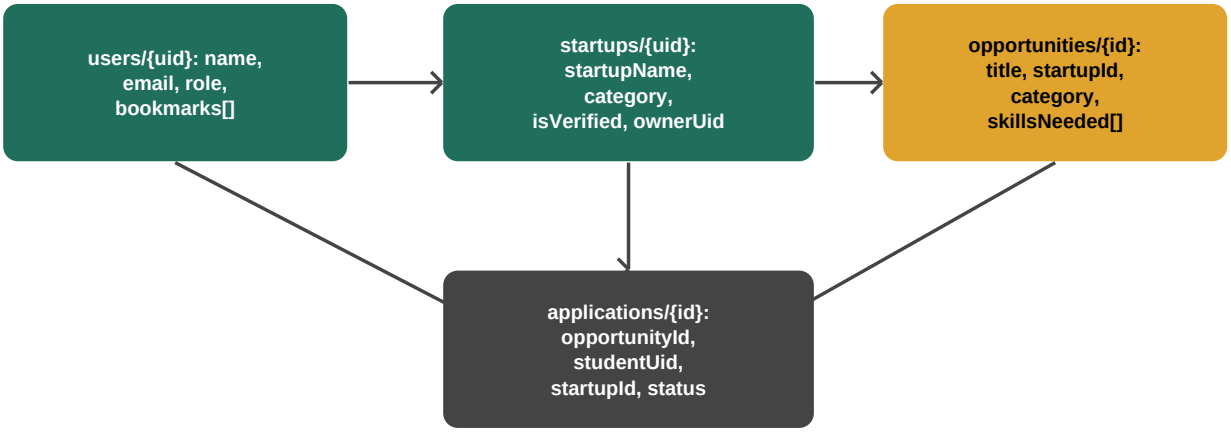


Figure 2. Firestore collections and how they reference each other.

Collection	Purpose	Key Fields
users/{uid}	One document per account	name, email, role, bookmarks (array)
startups/{uid}	Startup profile, id matches owner uid	startupName, description, category, isVerified, ownerUid
opportunities/{id}	One posted internship or gig	title, description, category, skillsNeeded, startupId, isOpen
applications/{id}	One student's application to one opportunity	opportunityId, studentUid, startupId, status, coverNote

3.1 Security Rules

Firestore security rules (see firestore.rules in the repository) enforce role based access at the database level, not just in the UI. Any signed-in user can read opportunities and startup profiles so students can browse, but a startup document can only be created by the account that owns it, and only an admin account can flip isVerified to true. Applications can only be created by the student who owns them, and

only the owning startup can update an application's status. This means even a modified client cannot bypass the verification workflow or read another student's private application data.

4. State Management Approach

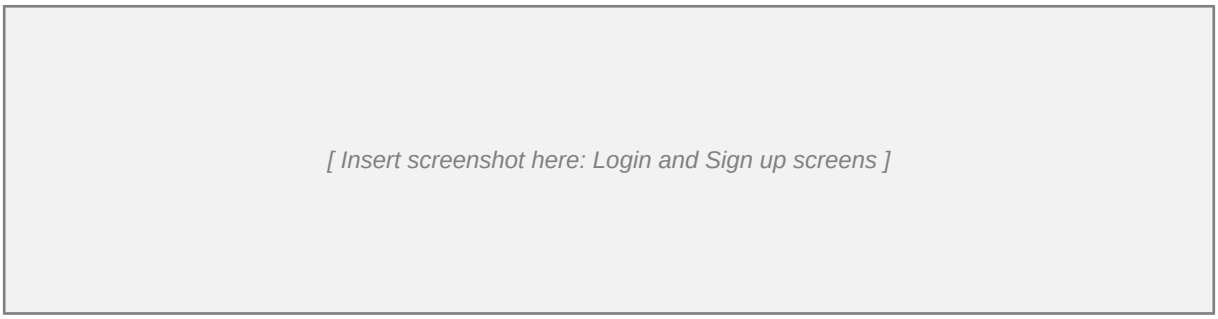
ALU Link uses the provider package, built on top of Flutter's InheritedWidget. Provider was chosen over alternatives such as BLoC or Riverpod because the app's state is mostly simple, list-based data coming straight from Firestore streams; provider lets that data reach any screen with very little boilerplate, while still being a recognized, production-grade state management approach. Each feature area (auth, opportunities, applications, startups, bookmarks) has its own provider instead of one large shared provider, which keeps each class focused on a single responsibility and makes it easy to find where a piece of state is managed.

State flows in one direction: a Firestore snapshot listener updates a provider's fields and calls notifyListeners(), any widget watching that provider rebuilds automatically, and user actions (such as tapping Apply or Post Opportunity) call a method on the provider, which writes to Firestore and lets the next snapshot event update the UI again. Screens never mutate provider fields directly.

5. Application Workflows

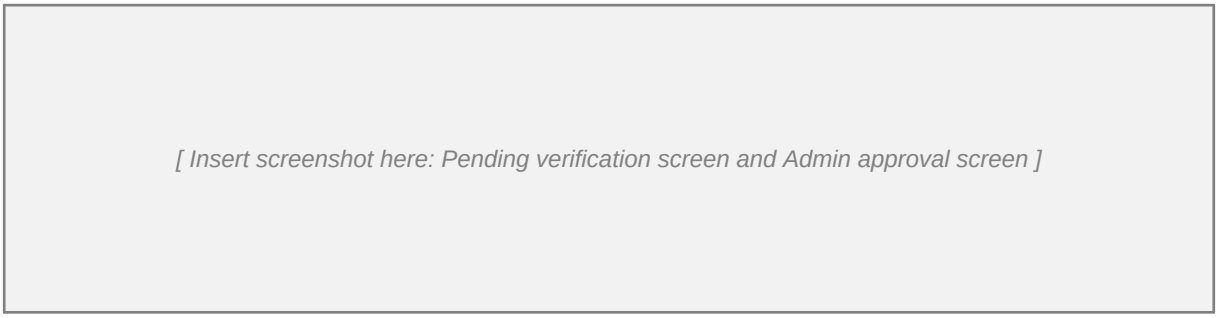
5.1 Authentication and Onboarding

A new user opens the app and chooses to sign up as either a Student or a Startup. This single choice determines everything that follows: students land directly on the opportunity browse screen, while startup accounts are sent to a profile setup form and then a pending verification screen until an admin approves them.



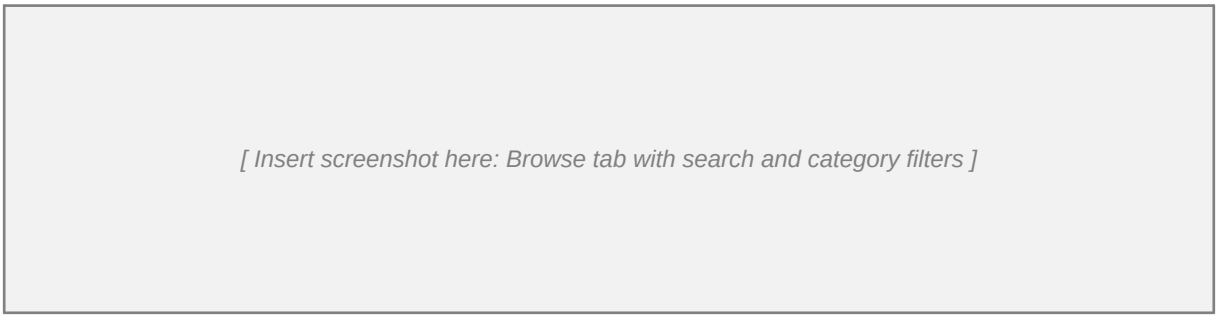
5.2 Startup Verification Workflow

After a startup account fills in its profile, the profile is saved with isVerified set to false and the founder sees a waiting screen. A separate admin account (configured in lib/constants.dart) sees every pending startup on a dedicated screen and can approve or reject each one. Only after approval do that startup's opportunities become visible to students, since the browse query filters out opportunities from startups that have not been verified.



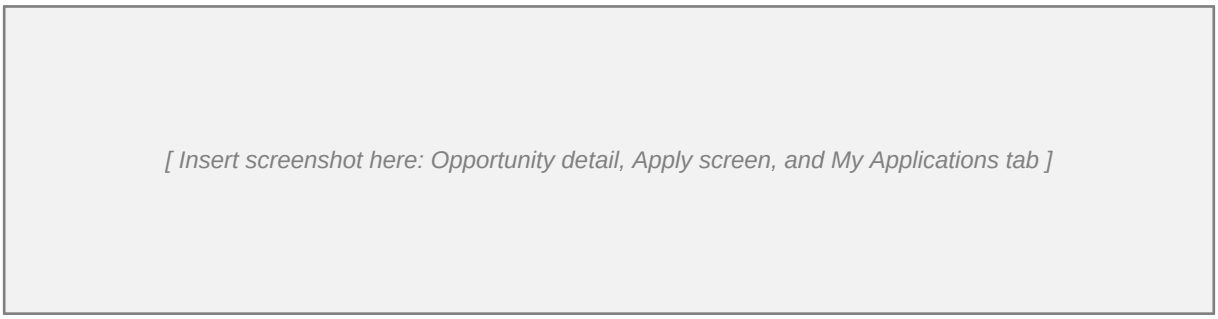
5.3 Opportunity Discovery and Search

Students browse opportunities on a live-updating list backed by a Firestore snapshot stream, so a newly posted opportunity appears without the student reloading the app. Students can type into a search box (matched against title and description) and tap category chips to filter by the same categories a startup chooses from when posting, keeping the two sides of the marketplace consistent.



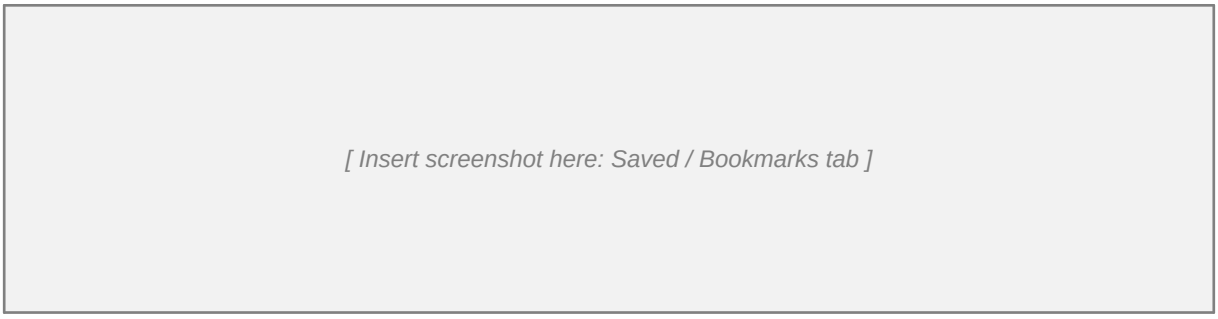
5.4 Application Submission and Tracking

From an opportunity's detail screen, a student can submit an application with a short cover note. The app blocks a second application to the same opportunity by checking Firestore before writing a new one. Every application the student has sent appears in a My Applications tab with a colored status chip (pending, accepted, or rejected) that updates live the moment the startup changes it.



5.5 Bookmarking

Students can save an opportunity for later by tapping a bookmark icon anywhere it appears. Saved opportunity ids are stored directly on the student's user document as a simple array, which keeps the bookmarking feature to one extra field instead of a whole new collection.



5.6 Startup Opportunity Management

Verified startups can post new opportunities with a title, description, category, required skills, and a remote or on-site flag. Each posting has its own applicants screen where the startup reviews every student who applied and accepts or rejects them, which immediately updates that student's application status.

[Insert screenshot here: Post Opportunity screen and Applicants screen]

6. UI and UX Design Reasoning

The color palette uses a deep teal and warm gold, chosen to feel distinct from a generic blue corporate job board and to read as approachable rather than purely administrative. A bottom navigation bar was used for the student home screen (Browse, Applications, Saved) because those three tasks are visited repeatedly in no fixed order, which is exactly the case bottom navigation is meant for, while the startup and admin sides use a simpler single-purpose screen since those users have fewer, more linear tasks.

Role-based routing (the AuthGate widget in main.dart) removes the need for the user to ever choose which part of the app to enter; a student only ever sees student screens and a startup only ever sees startup screens, which reduces the chance of a user landing on a screen that does not apply to them.

7. Scalability Considerations

- Firestore queries are structured around fields that are actually filtered on (startupId, studentUid, opportunityId, isVerified), so Firestore's automatic single-field indexes cover most reads without extra configuration.
- Client-side search and category filtering currently run over the full opportunity list loaded in memory. This is fine at the scale of one university's startup ecosystem, but a future version would move to a paginated query (using startAfter with createdAt) once the number of postings grows large.
- Each provider owns exactly one Firestore listener per screen it is responsible for, avoiding duplicate subscriptions and keeping bandwidth usage predictable as more users join.
- Security rules perform their checks using only the fields on the request and the existing document, so they do not require extra document reads and stay inexpensive as usage grows.

8. Testing Strategy

Automated tests (test/widget_test.dart) cover the parts of the app that do not require a live Firebase connection: a model test verifies that an Opportunity survives a round trip through toMap and fromMap the same way it would when read back from Firestore, and a widget test confirms the OpportunityCard widget renders a title, startup name, and a Remote tag correctly.

Beyond automated tests, the app was manually tested end to end on an emulator by walking through every workflow in section 5: signing up as both a student and a startup, approving a startup from the admin account, posting an opportunity, searching and filtering for it as a student, applying, and confirming the status updates on both the student and startup side in real time.

9. Challenges Encountered and Lessons Learned

(Fill this section in with your own experience once you have run the app end to end, for example: any Firestore rule you had to debug, any real-time listener that fired more often than expected, or any UI state that was tricky to keep in sync across screens. Keeping providers narrow and single-purpose, and

testing security rules directly in the Firebase console simulator, are two starting points worth expanding on.)

10. Limitations

- Startup verification is manual; there is no automated way to confirm a startup is officially recognized by ALU beyond admin review.
- There is no in-app messaging between students and startups after an application is accepted; contact currently happens outside the app.
- Search is a simple substring match on title and description, not a ranked or fuzzy search.
- There is no push notification support yet; status changes are only reflected while the app is open.

11. Future Improvements

- In-app chat between a startup and an accepted applicant.
- Push notifications (Firebase Cloud Messaging) for application status changes and new matching opportunities.
- A skill-matching recommendation feed that ranks opportunities by overlap with a student's listed skills.
- An analytics dashboard for startups showing applicant counts and category trends over time.
- Interview scheduling built into the applicant review screen.

References

Firebase. (2026). Cloud Firestore documentation. Google. https://firebase.google.com/docs/firestore

Firebase. (2026). Firebase Authentication documentation. Google. https://firebase.google.com/docs/auth

Flutter. (2026). Flutter documentation. Google. https://docs.flutter.dev/

Lomize, R. (2026). Provider package documentation (Version 6). pub.dev. https://pub.dev/packages/provider

Google. (2026). Material Design 3 guidelines. https://m3.material.io/